

Session 11 : Narrow Phase — SAT, GJK et Génération de Contact

Objectifs de la session

- Comprendre le rôle exact de la **Narrow Phase** dans le pipeline de collision.
- Maîtriser les **tests simples** : Sphère-Sphère, Sphère-AABB, AABB-AABB.
- Apprendre le **SAT (Separating Axis Theorem)** pour les polygones et boîtes orientées (OBB).
- Apprendre le **GJK (Gilbert-Johnson-Keerthi)** pour les formes convexes quelconques.
- Découvrir l'**EPA (Expanding Polytope Algorithm)** pour calculer la profondeur de pénétration.
- Générer un **contact complet** (point, normale, profondeur) prêt pour la Response.

🔔 Rappel de la Session 7

La Broad Phase a filtré les paires candidates avec des boîtes simplifiées (AABB). La Narrow Phase répond à la question : “**Ces deux objets se touchent-ils vraiment ?**” — et si oui, **comment** (point, normale, profondeur).

Partie 1 : Le Pipeline — Où se situe la Narrow Phase

Le pipeline en entonnoir

Un moteur physique moderne suit un **pipeline en entonnoir** qui élimine progressivement les faux positifs. Chaque étage divise le travail par un ordre de grandeur :

1. **Broad Phase** — “Qui pourrait entrer en collision ?” → 500 000 paires → 2 000 candidates.
2. **Narrow Phase** — “Se touchent-ils vraiment ?” → 2 000 candidates → 200 confirmées.
3. **Response** — “Que faire ?” → Impulsion + correction (vu en Session 6).

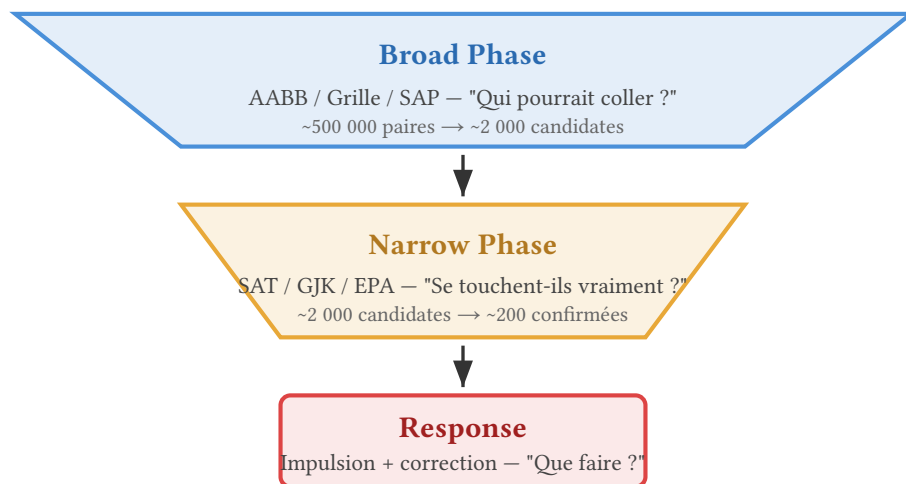


Figure 1: Le pipeline de détection de collision en entonnoir. La Broad Phase élimine 95–99 % des paires avec des tests triviaux. La Narrow Phase confirme les collisions avec des tests géométriques précis mais coûteux. La Response ne traite que les collisions réelles.

Le budget temps

À 60 FPS, la physique dispose de **2 ms** par frame. La Broad Phase consomme **< 0.5 ms**. La Narrow Phase et la Contact Generation ensemble doivent tenir dans le reste. C'est pourquoi on n'applique les tests coûteux (SAT, GJK) qu'aux paires **candidates** — pas aux N^2 paires.

Partie 2 : Hiérarchie des Tests Narrow Phase

Du moins cher au plus cher

On applique **toujours** les tests du moins cher au plus cher. Si un test simple échoue (pas de collision), on s'arrête — inutile de lancer GJK. C'est la même philosophie que l'entonnoir : chaque étage élimine des candidats à moindre coût.

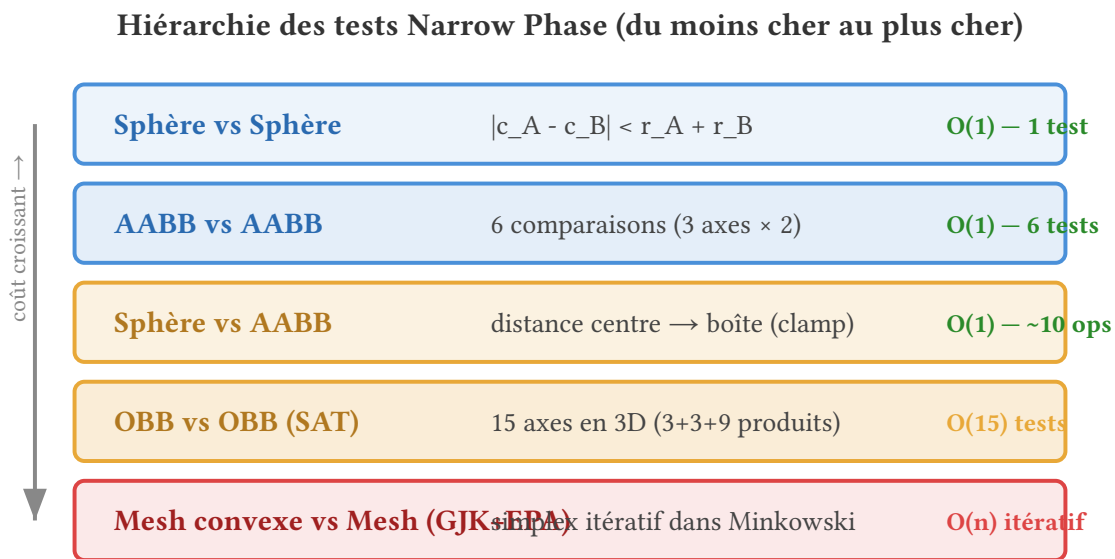


Figure 2: Hiérarchie des tests Narrow Phase, du moins cher (Sphère-Sphère, 1 comparaison) au plus cher (GJK+EPA pour meshes convexes). On remonte la hiérarchie uniquement si le test précédent n'a pas pu conclure.

Test 1 : Sphère vs Sphère — le plus simple

Principe

Deux sphères se touchent si la distance entre leurs centres est inférieure à la somme de leurs rayons :

$$\vec{d} = \vec{c}_B - \vec{c}_A$$

$$\text{collision} \Leftrightarrow \|\vec{d}\| < r_A + r_B$$

Pour éviter le calcul de racine carrée, on compare les carrés :

$$\vec{d} \cdot \vec{d} < (r_A + r_B)^2$$

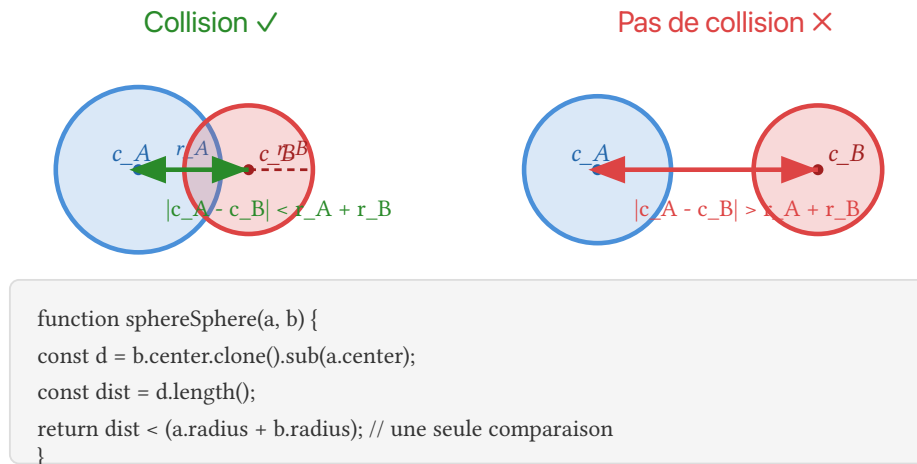


Figure 3: Test Sphère vs Sphère. À gauche : collision — la distance entre centres est inférieure à la somme des rayons. À droite : pas de collision — la distance est supérieure. Un seul test, $O(1)$.

💡 Optimisation : comparer les carrés

$\text{dist}^2 = dx*dx + dy*dy + dz*dz$ est beaucoup plus rapide que $\text{dist} = \text{sqrt}(dx*dx + dy*dy + dz*dz)$. On compare donc $\text{dist}^2 < (r_A + r_B)^2$. La racine carrée est une instruction lente — on l'évite toujours quand possible.

Test 2 : AABB vs AABB — déjà vu

Rappel (Session 7)

Deux AABB se chevauchent si et seulement si leurs intervalles se chevauchent sur **tous** les axes :

$$\text{overlap}_x = (x_{\max}^A \geq x_{\min}^B) \wedge (x_{\max}^B \geq x_{\min}^A)$$

Si un seul axe ne se chevauche pas, il n'y a pas de collision. **6 comparaisons en 3D.**

Test 3 : Sphère vs AABB — le clamp

Principe

On trouve le point de l'AABB le plus proche du centre de la sphère en **clampant** les coordonnées du centre sur les bornes de l'AABB :

$$\text{closest}_x = \text{clamp}(c_x, x_{\min}, x_{\max})$$

$$\text{closest}_y = \text{clamp}(c_y, y_{\min}, y_{\max})$$

$$\text{closest}_z = \text{clamp}(c_z, z_{\min}, z_{\max})$$

Puis on mesure la distance entre le centre et ce point :

$$\text{collision} \Leftrightarrow \|\vec{c} - \text{closest}\| < r$$

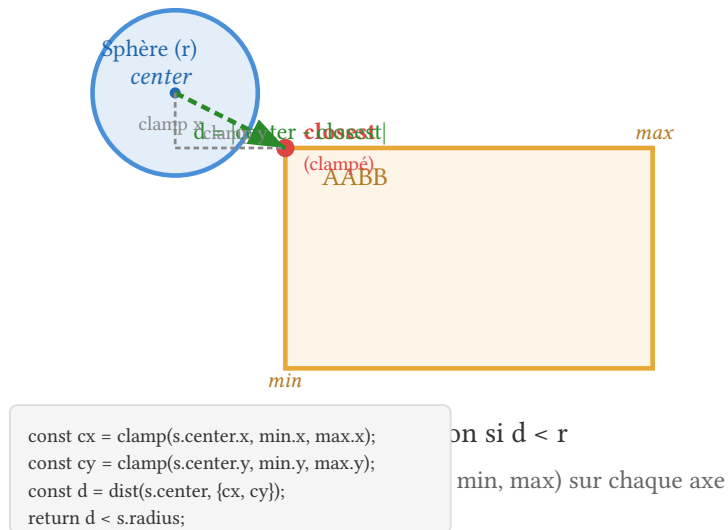


Figure 4: Test Sphère vs AABB. Le point le plus proche de l'AABB est obtenu en clampant le centre de la sphère sur chaque axe. Si la distance entre le centre et ce point clampé est inférieure au rayon, il y a collision.

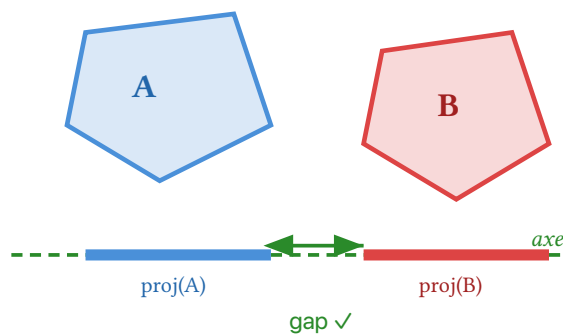
Partie 3 : SAT — Separating Axis Theorem

Le théorème

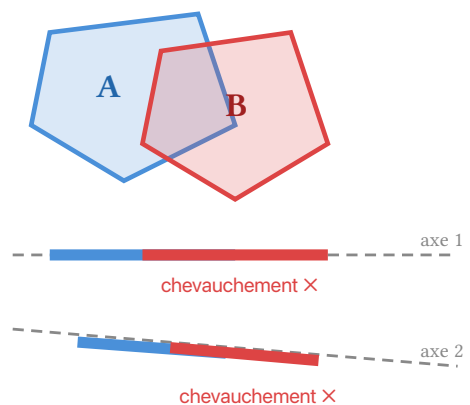
Si deux **polygones convexes** ne se touchent pas, alors il existe un axe (une droite) sur lequel leurs **projections** ne se chevauchent pas. Cet axe est appelé **axe séparateur**.

Inversement : si on teste tous les axes candidats et qu'aucun ne sépare les deux formes, alors **elles se touchent**.

Axe séparateur trouvé → pas de collision



Aucun axe séparateur → collision

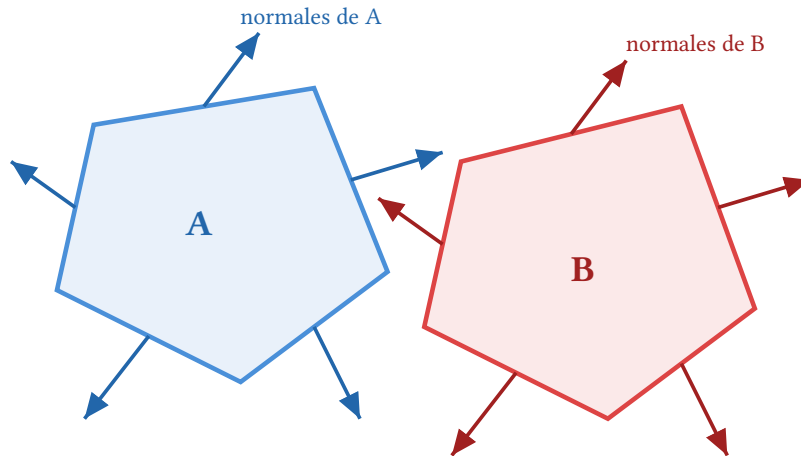


Tous les axes chevauchent → collision confir.

Figure 5: Principe du SAT. À gauche : un axe séparateur est trouvé — les projections de A et B sur cet axe ont un **gap** → pas de collision. À droite : tous les axes testés montrent un chevauchement → collision confirmée.

Quels axes tester ?

On ne teste pas **tous** les axes possibles (il y en a une infinité). On teste uniquement les **normales des faces** des deux polygones. Pour deux polygones convexes en 2D, cela fait $n_A + n_B$ axes (où n est le nombre de côtés). Si un seul de ces axes sépare les projections, c'est fini — pas de collision.



On teste chaque normale de face comme axe de projection. Si une seule sépare → pas de collision.

Figure 6: Les axes à tester pour SAT sont les normales des faces des deux polygones (en bleu pour A, en rouge pour B). On projette tous les sommets sur chaque axe et on regarde si les intervalles se chevauchent.

Comment projeter sur un axe

Projection d'un sommet

Pour projeter un sommet P sur un axe $\hat{n}$ (vecteur unitaire), on calcule le produit scalaire :

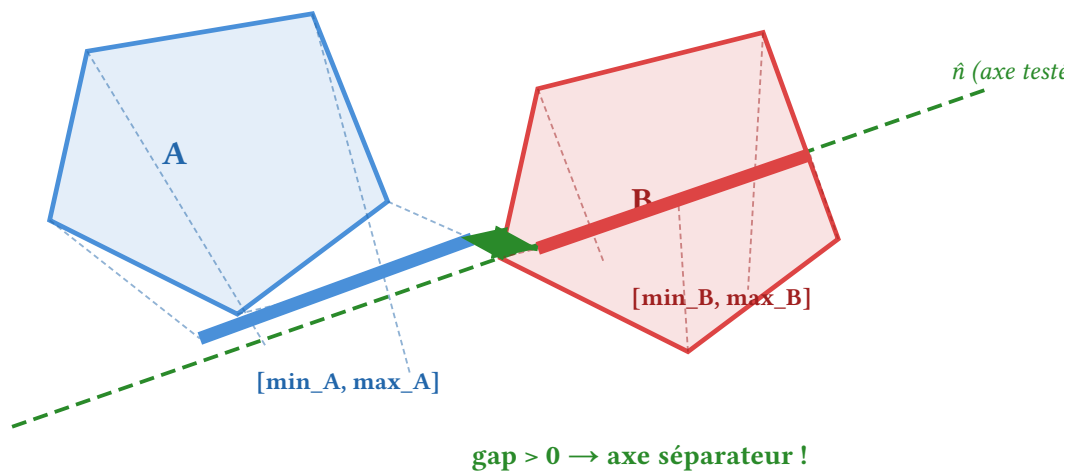
$$\text{proj}(P) = P \cdot \hat{n}$$

Pour un polygone entier, on projette **tous** ses sommets et on garde l'intervalle :

$$\text{intervalle} = [\min(\text{proj}(P_i)), \max(\text{proj}(P_i))]$$

Deux intervalles $[a_{\min}, a_{\max}]$ et $[b_{\min}, b_{\max}]$ se chevauchent si :

$$a_{\max} \geq b_{\min} \wedge b_{\max} \geq a_{\min}$$



Projection: $\text{proj}(P) = P \cdot \hat{n}$. Intervalle = $[\min(\text{proj}), \max(\text{proj})]$ sur tous les sommets.

Figure 7: Détail d'une projection SAT. Tous les sommets de A et B sont projetés sur l'axe $\hat{n}$. On obtient deux intervalles. Si un **gap** existe entre les intervalles, cet axe est séparateur — pas de collision.

L'algorithme SAT en résumé

SAT — algorithme

1. Collecter les axes candidats : normales de toutes les faces de A et B.
2. Pour chaque axe $\hat{n}$:
 - Projeter tous les sommets de A sur $\hat{n} \rightarrow$ intervalle $[a_{\min}, a_{\max}]$.
 - Projeter tous les sommets de B sur $\hat{n} \rightarrow$ intervalle $[b_{\min}, b_{\max}]$.
 - Si les intervalles **ne se chevauchent pas** $\rightarrow$ **axe séparateur trouvé** $\rightarrow$ pas de collision, on s'arrête.
3. Si **aucun** axe ne sépare $\rightarrow$ collision confirmée.

L'axe avec le **plus petit chevauchement** donne la **normale de contact** et la **profondeur de pénétration** — gratuitement !

💡 Bonus : contact gratuit

SAT ne dit pas juste "oui/non". L'axe avec le **plus petit chevauchement** de projection est la **normale de contact**, et la valeur de ce chevauchement est la **profondeur de pénétration**. Pas besoin d'EPA — SAT donne le contact directement.

SAT en 3D : OBB vs AABB

Oriented Bounding Box (OBB)

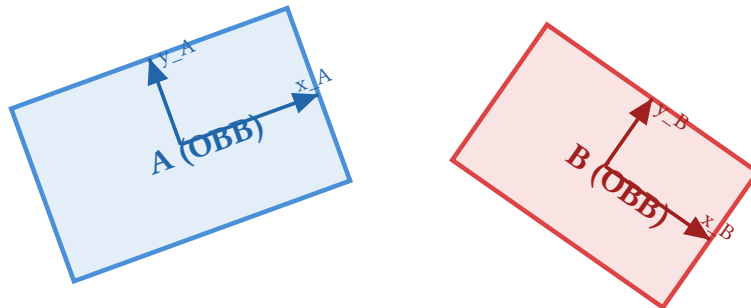
Une OBB est comme une AABB, mais **orientée** — elle peut être tournée. Elle suit les axes **locaux** de l'objet, pas les axes du monde. Plus précise qu'une AABB pour les objets allongés et tournés (un fusil, une poutre).

! 15 axes en 3D

Pour deux OBB en 3D, SAT doit tester **15 axes** :

- **3 normales de faces de A** (x_A, y_A, z_A — les axes locaux de A).
- **3 normales de faces de B** (x_B, y_B, z_B — les axes locaux de B).
- **9 produits vectoriels** ($x_A \times x_B, x_A \times y_B, \dots$ — une pour chaque combinaison d'arêtes).

Si un seul de ces 15 axes sépare les projections $\rightarrow$ pas de collision. Sinon $\rightarrow$ collision, et l'axe avec le plus petit chevauchement donne le contact.



En 3D, SAT teste 15 axes pour OBB vs OBB :

- 3 normales de faces de A (x_A, y_A, z_A)
 - 3 normales de faces de B (x_B, y_B, z_B)
 - 9 produits vectoriels ($x_A \times x_B, x_A \times y_B, \dots$)
- Total = 3 + 3 + 9 = 15 axes à tester

Figure 8: OBB vs OBB : deux boîtes orientées différemment. SAT teste 15 axes en 3D — les 3 normales de faces de chaque boîte, plus 9 produits vectoriels d'arêtes. C'est plus coûteux que l'AABB (6 comparaisons) mais reste direct et non itératif.

Limites de SAT

! Quand SAT ne suffit pas

SAT fonctionne uniquement pour des formes **convexes**. Un polygone concave (en L, en U, en étoile) ne peut pas être testé avec SAT directement — il faut décomposer la forme en sous-formes convexes (convex decomposition), ce qui est coûteux. Pour les formes convexes **quelconques** (meshes convexes, capsules, cônes), on utilise **GJK** à la place.

Partie 4 : GJK — Gilbert-Johnson-Keerthi

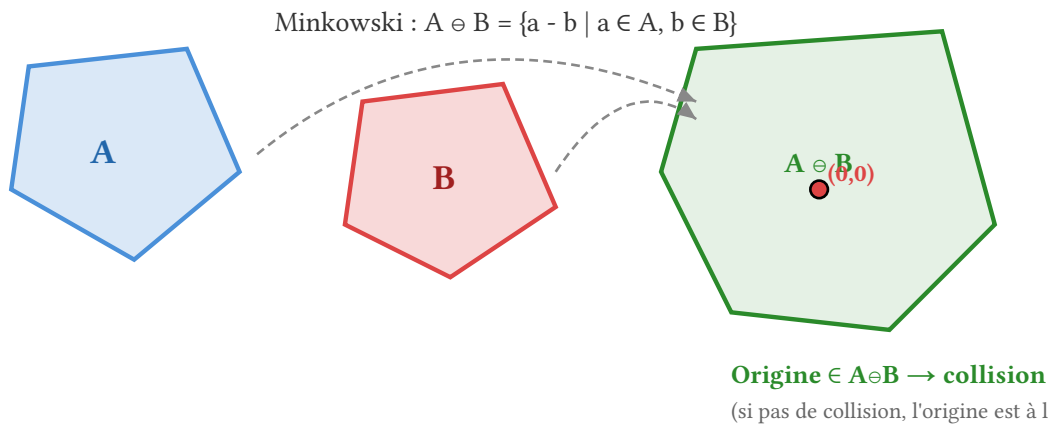
Le principe de GJK

GJK travaille dans l'**espace de Minkowski**. Au lieu de comparer directement deux formes A et B, on construit la **différence de Minkowski** :

$$A \ominus B = \{ \vec{a} - \vec{b} \mid \vec{a} \in A, \vec{b} \in B \}$$

Propriété clé : A et B se touchent si et seulement si l'origine (0, 0, 0) est à l'**intérieur** de $A \ominus B$.

GJK ne construit pas $A \ominus B$ explicitement (ce serait trop coûteux). Il l'**échantillonne** itérativement avec la **fonction de support**.



GJK ne construit pas $A \ominus B$ explicitement — il échantillonne avec la fonction de support.

Figure 9: La différence de Minkowski $A \ominus B$. Si l'origine est à l'intérieur de cette forme, A et B sont en collision. GJK n'a pas besoin de construire toute la forme — il l'échantillonne avec la fonction de support.

La fonction de support

Fonction de support

La fonction de support $S_A(\vec{d})$ retourne le point de A le plus extrême dans la direction $\vec{d}$:

$$S_A(\vec{d}) = \operatorname{argmax}_{P \in A} (P \cdot \vec{d})$$

C'est-à-dire : **“Quel est ton point le plus loin dans la direction $\vec{d}$?”**

Pour la différence de Minkowski :

$$S_{A \ominus B}(\vec{d}) = S_A(\vec{d}) - S_B(-\vec{d})$$

On n'a besoin que des fonctions de support de A et B séparément — jamais de construire $A \ominus B$.

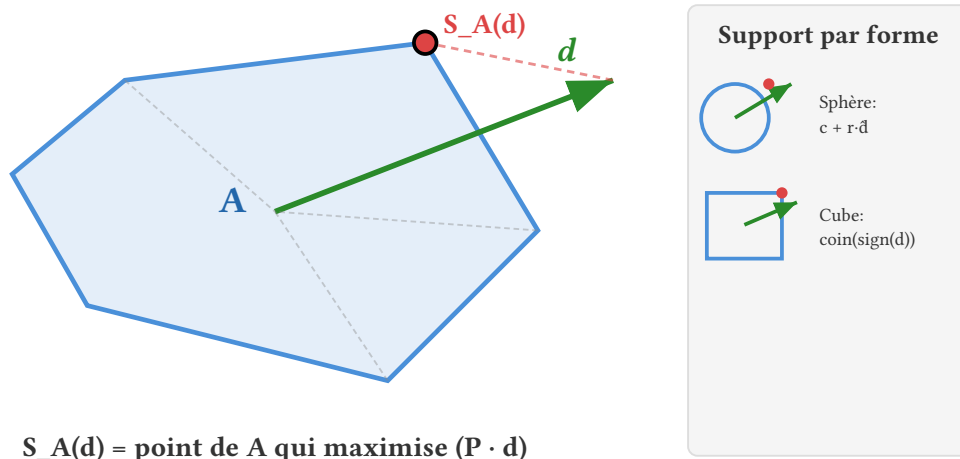


Figure 10: La fonction de support. Pour une direction $\vec{d}$ donnée, $S_A(\vec{d})$ est le sommet de A qui maximise le produit scalaire avec $\vec{d}$. Pour une sphère, c'est $\vec{c} + r \cdot \hat{\vec{d}}$. Pour un cube, c'est le coin correspondant au signe des composantes de $\vec{d}$.

💡 Support par forme

La fonction de support est **triviale** pour les formes primitives :

- **Sphère** : $S(\vec{d}) = \vec{c} + r \cdot \hat{\vec{d}}$ — le centre plus le rayon dans la direction.
- **AABB / OBB** : le coin correspondant au signe des composantes de $\vec{d}$.
- **Mesh convexe** : parcourir tous les sommets et garder celui avec le plus grand produit scalaire — $O(n)$ mais on peut précalculer.

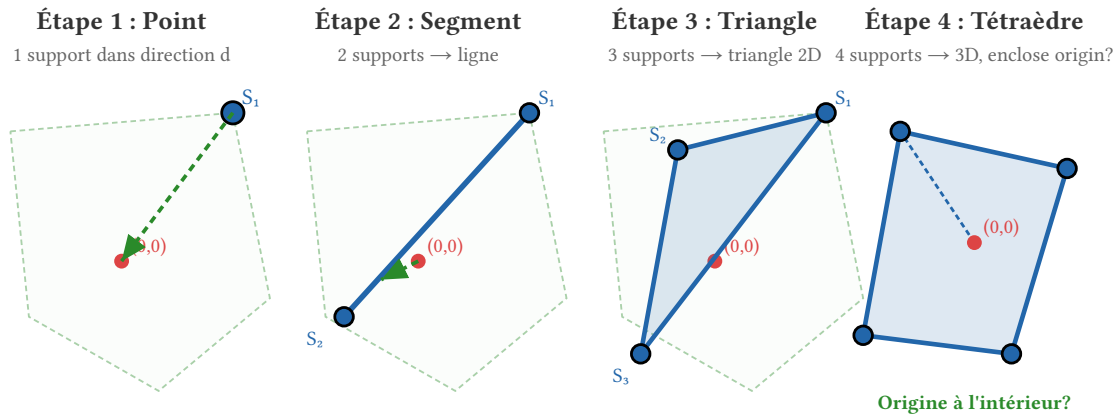
L'algorithme GJK — construire un simplex

Simplex

Un **simplex** est la généralisation d'un triangle en toute dimension :

- 0D : un **point**.
- 1D : un **segment** (2 points).
- 2D : un **triangle** (3 points).
- 3D : un **tétraèdre** (4 points).

GJK construit itérativement un simplex dans $A \ominus B$ qui **encercler l'origine**. Si le simplex finit par contenir l'origine $\rightarrow$ collision. Si on ne peut plus l'étendre $\rightarrow$ pas de collision.



Algorithme GJK :

1. Support initial $\rightarrow$ 2. Étendre le simplex vers l'origine $\rightarrow$ 3. L'origine est-elle contenue ?
Si oui $\rightarrow$ collision. Si non (le simplex ne peut plus grandir) $\rightarrow$ pas de collision.

Figure 11: Évolution du simplex dans GJK. Étape 1 : un point de support. Étape 2 : un segment. Étape 3 : un triangle (2D). Étape 4 : un tétraèdre (3D). À chaque étape, on étend le simplex **vers l'origine**. Si l'origine finit à l'intérieur $\rightarrow$ collision.

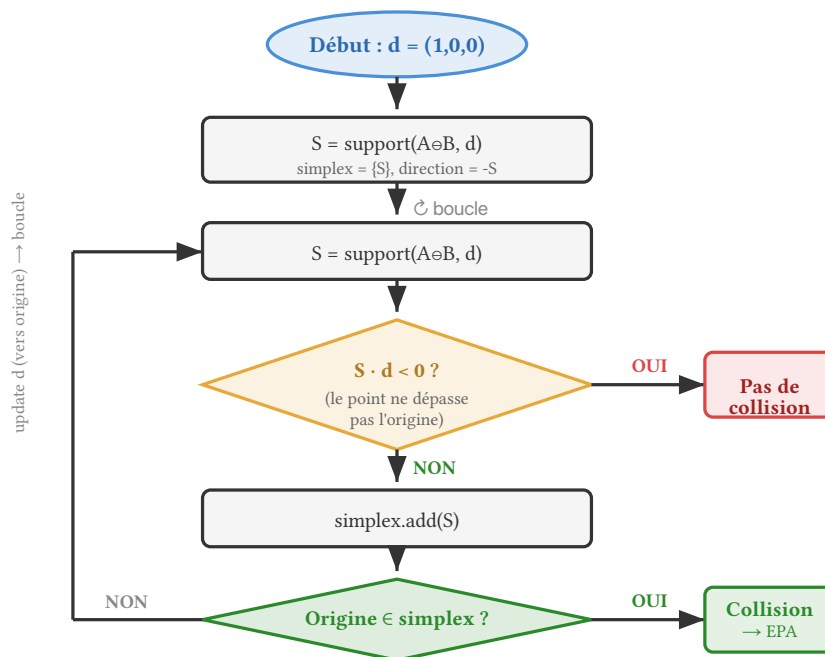


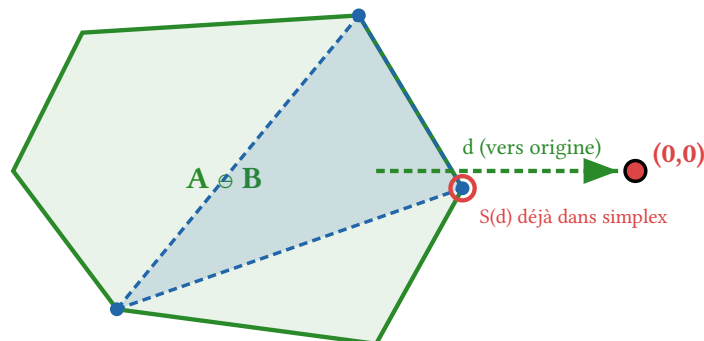
Figure 12: Organigramme de l'algorithme GJK. On obtient un point de support, on vérifie s'il dépasse l'origine (sinon, pas de collision), on l'ajoute au simplex, on vérifie si l'origine est contenue (si oui, collision), sinon on met à jour la direction et on boucle.

GJK : cas sans collision

Condition d'arrêt — pas de collision

À chaque itération, on demande le support dans la direction de l'origine. Si ce nouveau point **ne dépasse pas l'origine** sur cette direction (c'est-à-dire $S \cdot \vec{d} < 0$), cela signifie que l'origine est **hors de portée** — il n'y a **pas de collision**.

C'est la beauté de GJK : il peut conclure "pas de collision" dès qu'il n'y a plus de point à échantillonner dans la bonne direction.



rt dans la direction de l'origine est déjà dans le simplex → l'origine est hors $A \ominus B$ → pas d

Figure 13: GJK sans collision. L'origine est hors de $A \ominus B$. Le simplex ne peut plus grandir vers l'origine — le support dans la direction de l'origine est déjà dans le simplex. GJK conclut : pas de collision.

GJK : cas avec collision → EPA

💡 GJK dit oui, mais pas combien

GJK répond à la question "Est-ce qu'il y a collision ?" — oui ou non. Mais il ne dit pas **la profondeur de pénétration** ni **la normale de contact**. Pour obtenir ces informations, on enchaîne avec EPA (Expanding Polytope Algorithm).

Partie 5 : EPA — Expanding Polytope Algorithm

Principe d'EPA

Une fois que GJK a confirmé la collision (le simplex contient l'origine), EPA **étend** ce simplex vers la vraie frontière de $A \ominus B$:

1. On prend le simplex final de GJK (triangle en 2D, tétraèdre en 3D).
2. On trouve l'arête/face la plus proche de l'origine.
3. On demande le support dans la direction de la normale de cette arête/face.
4. Si le nouveau point est plus loin que l'arête/face, on l'ajoute au polytope (qui grandit).
5. On répète jusqu'à ne plus pouvoir grandir — l'arête/face la plus proche de l'origine donne la **normale de contact** et la **profondeur de pénétration**.

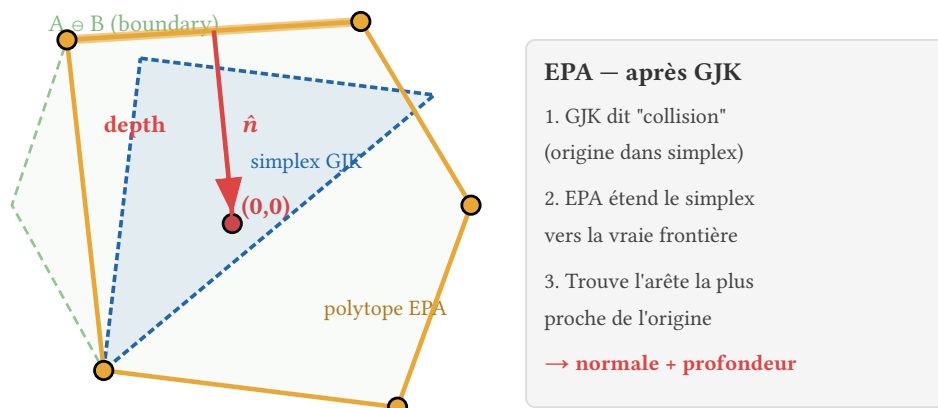


Figure 14: EPA après GJK. Le simplex de GJK (bleu pointillé) contient l'origine. EPA étend le polytope (orange) vers la vraie frontière de $A \oplus B$. L'arête la plus proche de l'origine donne la normale $\hat{n}$ (rouge) et la profondeur de pénétration.

💡 EPA = le complément de GJK

GJK et EPA travaillent en tandem :

- **GJK** : "Y a-t-il collision ?" → OUI/NON.
- **EPA** : "Si oui, quelle est la normale et la profondeur ?"

Ensemble, ils forment le duo standard pour la détection de collision sur des formes convexes quelconques dans les moteurs professionnels (Bullet, PhysX, Box2D).

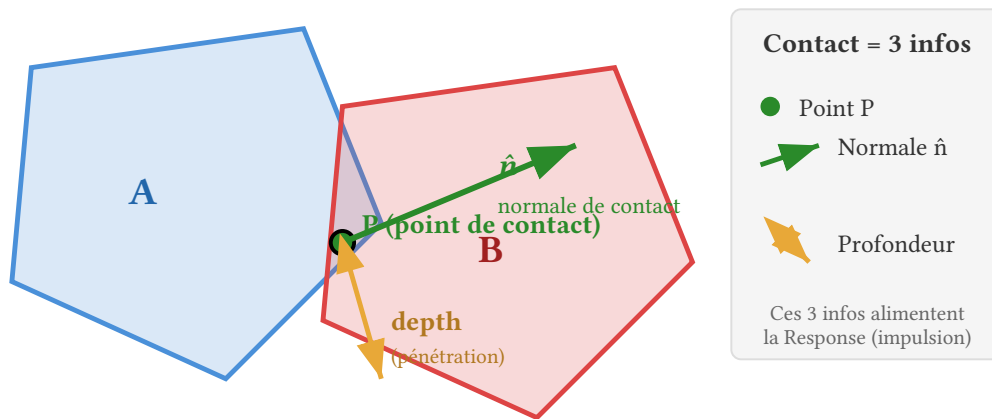
Partie 6 : Génération du Contact

Les trois ingrédients d'un contact

Une fois la collision confirmée (par SAT, GJK+EPA, ou un test simple), la Narrow Phase doit produire un **contact** avec trois informations :

1. **Point de contact** P — où les deux objets se touchent (en coordonnées du monde).
2. **Normale de collision** $\hat{n}$ — direction dans laquelle séparer les objets (de A vers B).
3. **Profondeur de pénétration** d — de combien les objets se chevauchent.

Ces trois valeurs alimentent directement la **Response** (Session 6) : l'impulsion est calculée le long de $\hat{n}$, la correction de position déplace les objets de d .



La Narrow Phase ne dit pas juste "oui/non" — elle calcule le contact complet.

Sans ces 3 valeurs, la Response (Session 6) ne peut pas appliquer d'impulsion.

Figure 15: Un contact complet : le point P (vert), la normale $\hat{n}$ (vert, de A vers B), et la profondeur de pénétration (orange). Sans ces trois valeurs, la Response ne peut pas appliquer d'impulsion ni corriger la position.

Contact selon la méthode

Méthode	Point	Normale	Profondeur
Sphère-Sphère	milieu du segment	direction centres	somme rayons - distance
Sphère-AABB	closest point	centre - closest	rayon - distance
AABB-AABB	centre du chevauchement	axe de pénétration min	chevauchement min
SAT	sur la face	axe séparateur min	chevauchement min
GJK + EPA	calculé par EPA	EPA arête closest	distance arête-origine

Table 1: Comment chaque méthode produit le contact (point, normale, profondeur). Les tests simples donnent le contact directement. SAT donne la normale et la profondeur via l'axe de chevauchement minimal. GJK a besoin d'EPA pour obtenir ces informations.

Partie 7 : SAT vs GJK – Quand utiliser quoi

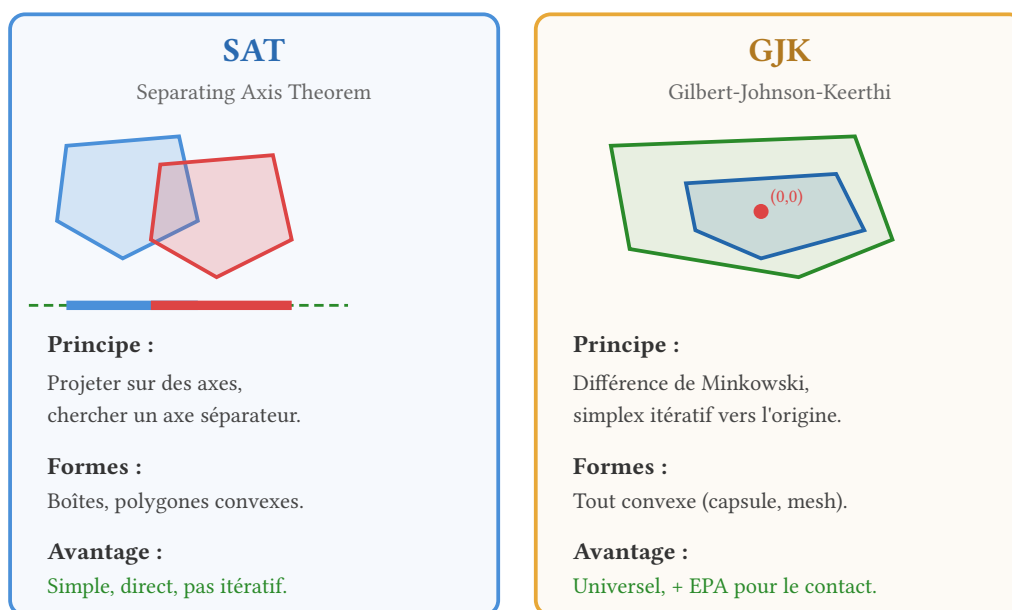


Figure 16: Comparaison SAT vs GJK. SAT projette sur des axes fixes et cherche un séparateur — simple et direct. GJK construit un simplex itératif dans l'espace de Minkowski — universel mais itératif.

Critère	SAT	GJK
Formes supportées	Boîtes, polygones convexes	Tout convexe (mesh, capsule, cône)
Dimension	2D : $n_A + n_B$ axes, 3D : 15 axes (OBB)	Itératif, 2–4 itérations typique
Coût par paire	$O(n)$ (nombre de sommets)	$O(k)$ itérations $\times$ coût support
Contact direct	Oui (axe min chevauchement)	Non (besoin d'EPA)
Convexes quelconques	Non (axes fixes)	Oui
Implementation	Simple	Plus complexe
Cas d'usage	OBB vs OBB, polygones 2D	Mesh convexe, capsule, formes custom

Table 2: Comparaison détaillée SAT vs GJK. SAT est plus simple et donne le contact directement, mais limité aux boîtes et polygones. GJK est universel pour toute forme convexe mais nécessite EPA pour le contact.

💡 En pratique dans les moteurs

La plupart des moteurs physiques utilisent **les deux** :

- **SAT** pour les OBB (boîtes orientées) — c'est le cas le plus fréquent.
- **GJK + EPA** pour les meshes convexes et formes custom (capsules, cylindres, cônes).

- **Tests simples** (sphère-sphère, sphère-AABB) pour les cas triviaux — la majorité des collisions dans un jeu.

On commence toujours par le test le moins cher. Si on a des sphères, on teste sphère-sphère. Si on a des boîtes orientées, on teste OBB-OBB avec SAT. Si on a des meshes convexes, on passe à GJK.

Partie 8 : Le Pipeline Complet

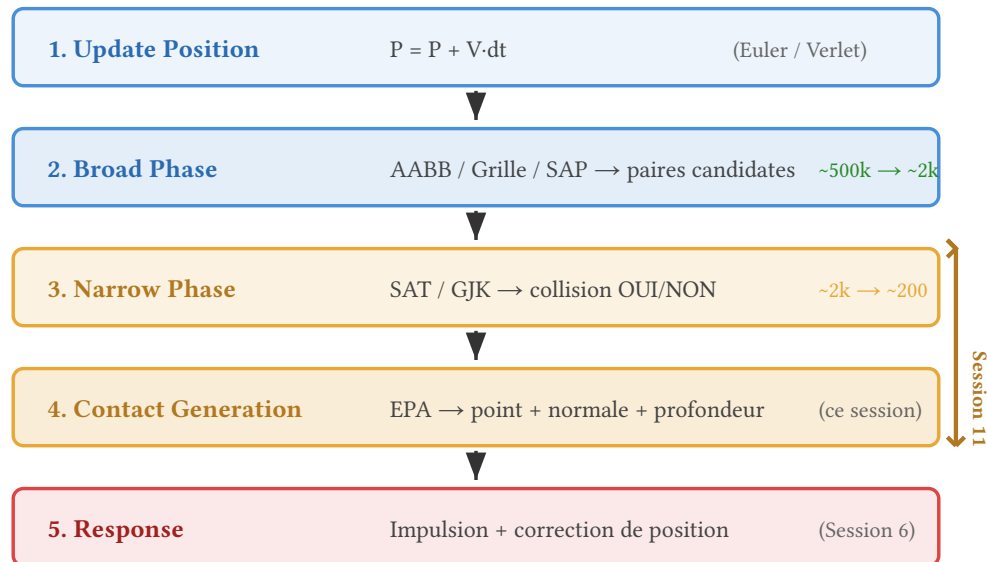


Figure 17: Le pipeline complet de détection et résolution de collision. La Session 11 couvre les étapes 3 (Narrow Phase) et 4 (Contact Generation). L'étape 5 (Response) a été vue en Session 6.

Résumé du pipeline

1. **Update Position** : $P = P + V \cdot dt$ (Euler ou Verlet).
2. **Broad Phase** : AABB / Grille / SAP → paires candidates (500k → 2k).
3. **Narrow Phase** : Sphère-Sphère / AABB / SAT / GJK → collision OUI/NON (2k → 200).
4. **Contact Generation** : point + normale + profondeur (EPA ou axe SAT minimal).
5. **Response** : impulsions pour séparer les objets + correction de position (Session 6).

Partie 9 : Code — SAT en JavaScript (2D)

SAT pour deux polygones convexes 2D.

```

function sat(polyA, polyB) {
  const axes = [...getNormals(polyA), ...getNormals(polyB)];
  let minOverlap = Infinity;
  let minAxis = null;

  for (const axis of axes) {
    // Projeter tous les sommets de A sur l'axe
    const [minA, maxA] = project(polyA, axis);
    // Projeter tous les sommets de B sur l'axe
    const [minB, maxB] = project(polyB, axis);

    // Y a-t-il un gap ?
    if (maxA < minB || maxB < minA) {

```

```

    // Axe séparateur trouvé → pas de collision
    return null;
}

// Mesurer le chevauchement sur cet axe
const overlap = Math.min(maxA - minB, maxB - minA);
if (overlap < minOverlap) {
    minOverlap = overlap;
    minAxis = axis;
}
}

// Aucun axe séparateur → collision
// minAxis = normale de contact, minOverlap = profondeur
return { normal: minAxis, depth: minOverlap };
}

function project(poly, axis) {
    let min = Infinity, max = -Infinity;
    for (const v of poly.vertices) {
        const proj = v.dot(axis);
        min = Math.min(min, proj);
        max = Math.max(max, proj);
    }
    return [min, max];
}

```

Partie 10 : Code — GJK en JavaScript (2D)

GJK pour deux formes convexes (support function requise).

```

function gjk(supportA, supportB) {
    let d = new Vector2(1, 0); // direction initiale arbitraire
    let simplex = [support(supportA, supportB, d)];
    d = simplex[0].clone().negate(); // vers l'origine

    for (let i = 0; i < 50; i++) { // limite d'itérations
        const newPoint = support(supportA, supportB, d);

        // Le point ne dépasse pas l'origine → pas de collision
        if (newPoint.dot(d) < 0) return null;

        simplex.push(newPoint);

        // L'origine est-elle dans le simplex ?
        if (containsOrigin(simplex)) {
            return { simplex }; // collision → passer à EPA
        }

        // Mettre à jour d : direction vers l'origine
        // depuis le point/segment/triangle le plus proche
        d = closestDirectionToOrigin(simplex);
        // Retirer les points du simplex qui ne contribuent pas
        simplex = reduceSimplex(simplex, d);
    }
    return null; // trop d'itérations → abandon
}

```



```
// Support de  $A \oplus B = S_A(d) - S_B(-d)$ 
function support(supportA, supportB, d) {
    return supportA(d).sub(supportB(d.clone()).negate()));
}
```

⚠ GJK est subtil à implémenter

Les parties difficiles de GJK sont `containsOrigin` (vérifier si l'origine est dans le simplex) et `closestDirectionToOrigin` (mettre à jour la direction). En 2D, c'est gérable avec un triangle. En 3D avec un tétraèdre, les cas à traiter sont nombreux. C'est pourquoi la plupart des jeux utilisent une bibliothèque physique (Bullet, Cannon.js, Rapier) plutôt que de réimplémenter GJK.

Synthèse

💬 Ce qu'il faut retenir

- La **Narrow Phase** confirme les collisions entre paires candidates et génère le **contact** (point, normale, profondeur).
- On applique les tests du **moins cher au plus cher** : Sphère-Sphère → AABB → SAT → GJK.
- **SAT** projette sur les normales des faces — simple, direct, donne le contact gratuitement. Limité aux boîtes et polygones convexes.
- **GJK** construit un simplex dans l'espace de Minkowski — universel pour toute forme convexe, mais itératif et ne donne pas le contact directement.
- **EPA** étend le simplex de GJK pour trouver la normale et la profondeur de pénétration.
- Le **contact** (point + normale + profondeur) alimente la **Response** (impulsion + correction, Session 6).